Lab Exercise: Uncertainty in Estimation - Testing Prejudice in Football

Todd K Hartman

Last updated 2024-03-13

Getting Started

In this lab exercise, we're going to test whether there is any evidence that racial and/or ethnic bias affects whether players are penalized in sports. Twenty-nine research teams analyzed the same dataset to investigate the research question: are football referees more likely to give red cards to players with darker skin tone than to those with lighter skin tone? This "crowdsourced" approach to research revealed that there were numerous analytic approaches used, which resulted in estimated effect sizes ranging from 0.89 to 2.93 (Median = 1.31) in odds-ratio units (i.e., the likelihood that a player with dark skin tone would receive a penalty relative to one with light skin tone; an odds ratio of 1.31 means that the former were 1.31 times more likely to be penalized compared to the latter). Ultimately, these diverse findings suggest that "significant variation in the results of analyses of complex data may be difficult to avoid, even by experts with honest intentions".

This practical is based on the following published article:

Silberzahn, R., Uhlmann, E.L., Martin, D.P., Anselmi, P., Aust, F., Awtrey, E., Bahník, Š., Bai, F., Bannard, C., Bonnier, E. and Carlsson, R., 2018. Many analysts, one data set: Making transparent how variations in analytic choices affect results. Advances in Methods and Practices in Psychological Science, 1(3), pp.337-356. https://doi.org/10.1177/2515245917747646

The Football Data

The dataset contains profiles (including photos) from male football players (N = 2,053) playing in the highest divisions of England, Germany, France, and Spain in the 2012-2013 season, as well as all referees (N = 3,147) during the players' professional careers. Thus, the dataset contains 146,028 player-referee combinations, or *dyads*. The outcome of interest (dependent variable) is the number of penalties given to a player by a particular referee throughout all matches the two encountered each other. We'll convert this count into a rate to make it comparable across players (i.e., if we used the raw counts, then individuals who played more games would have more opportunities to be penalised).

Player photos were available for 77% (1,586/2,053) of players. Players' skin tone was coded by two independent raters who categorized players on a 5-point scale ranging from 1 = "very light skin" to 5 = "very dark skin" (the midpoint = "neither dark nor light skin").

In addition, (implicit) bias scores for each referee country were calculated using a race Implicit Association Test (IAT), with higher values corresponding to faster white-good, black-bad associations. The idea is that people should respond faster to concepts that are associated in memory and slower for ones that are not associated. Explicit bias scores for each referee country were calculated using a racial thermometer task, with higher values corresponding to greater feelings of warmth toward whites versus blacks. Both these measures were created by aggregating data from many online users in referee countries taking these tests on Project Implicit (http://projectimplicit.net). Note that these bias measures were not created from representative, probability-based samples so you should use caution when generalizing to the population.

A detailed description of all variables in the dataset can be seen in the list below.

Variable	Description
playerShort	short player ID
player	player name
club	player club
leagueCountry	country of player club (England, Germany, France, and Spain)
birthday	player birthday
height	player height (in cm)
weight	player weight (in kg)
position	detailed player position
games	number of games in the player-referee dyad
victories	victories in the player-referee dyad
ties	ties in the player-referee dyad
defeats	losses in the player-referee dyad
goals	goals scored by a player in the player-referee dyad
yellowCards	number of yellow cards player received from referee
yellowReds	number of yellow-red cards player received from referee
redCards	number of red cards player received from referee
photoID	ID of player photo (if available)
rater1	skin rating of photo by rater 1 (5-point scale ranging from "very light skin" to "very dark skin")
rater2	skin rating of photo by rater 2 (5-point scale ranging from "very light skin" to "very dark skin")
refNum	unique referee ID number (referee name removed for anonymizing purposes)
refCountry	unique referee country ID number (country name removed for anonymizing purposes)
meanIAT	mean implicit bias score (using the race IAT) for referee country, higher values correspond to faster white
nIAT	sample size for race IAT in that particular country
seIAT	standard error for mean estimate of race IAT
meanExp	mean explicit bias score (using a racial thermometer task) for referee country, higher values correspond to greater feelings of warmth toward whites versus blacks
nExp	sample size for explicit bias in that particular country
seExp	standard error for mean estimate of explicit bias measure

Loading the necessary packages

Let's begin by setting the working directory and loading any necessary packages (e.g. tidyverse).

```
## Make sure the 'pacman' package manager is installed the first time!
# Uncomment if necessary to install pacman
# install.packages('pacman')

## Load packages via 'pacman' package manager
## pacman:: tells R to use the function 'p_load' from the 'pacman' library
pacman::p_load(lubridate, magrittr, sjPlot, tidyverse)

## Don't forget to set your working directory!
setwd("ENTER YOUR WORKING DIRECTORY HERE")
```

Getting the data

The data is available at the Open Science Framework (OSF). We're going to *automagically* download, unzip, and load the file into R. If you have trouble with this step, just manually download the file, unzip it, and

read the .csv into R as you would normally do. Note that we're going to use the `read_csv()` function from `tidyverse` so that our data is a *tibble*, which is a *tidy* dataframe.

```
## Download and import from the web (or download manually)
url.zip <- "https://osf.io/fv8c3/download"  # Download location
file.zip <- "1. Crowdsourcing Dataset July 01, 2014 Incl.Ref Country.zip"  # File name
if (!file.exists(file.zip))  # If the file is not in the working directory...
    download.file(url = url.zip, destfile = file.zip, mode = "wb")  # ...then download the file
file.csv <- grep("csv", unzip(file.zip),
                 ignore.case = TRUE, value = TRUE)  # Unzip and grab filename

fb <- read_csv(file.csv[1])  # Load the data

## If all else fails, load the .RData version directly! [Only as a last resort]
# load(football.RData)  # Uncomment if needed
```

Checking the data

Now let's check the data to ensure that we've done everything correctly. For today's practical, we're going to focus on each player as the level of analysis, which means that we have to get these 146,028 player-referee dyads into a dataset with 2,053 rows for analysis. In other words, the data is currently organized into a *long* format with repeated observations for each player. It's called "long" because the shape of the data looks like a vertically-oriented rectangle. Long datasets have repeated measures for each observation as we can see in this example below (i.e., multiple rows for the same ID number):

ID	Weight
1	72.1 kg
1	71.8 kg
1	70.6 kg
2	49.9 kg
2	50.1 kg
2	50.8 kg

Instead, we want the data in a *wide* format, which means that there is only one row for each unique observation; repeated measures will show up in columns as we can see below:

ID	Weight1	Weight 2	Weight3
1	72.1 kg	71.8 kg	70.6 kg
2	49.9 kg	50.1 kg	50.8 kg

How might we transform the data from long to wide?

```
glimpse(fb)  # Check the data
```

Transforming the data for analysis

As you might have guessed, there are many ways that we could accomplish this task. One approach would be to use the `aggregate` function to group the data by player. This function takes 3 arguments: the variable to aggregate and the grouping variable, the dataset, and the function to apply during aggregation. For some of the measures, we want to sum them (e.g., total yellow and red cards). For others, we just want the unique value since it doesn't change (e.g., each rating of a player's skin tone is a single measure but is repeated many

times because of the long format of the data; height and weight are other examples of the same value that is just repeated - if we ask for the mean, R will provide us the unique value).

For now, let's just transform the data for penalties (yellow, yellow-reds, and red cards), games played, and skin tone ratings.

```
## Use the 'aggregate()' function to collapse the data for key measures
## Outcome variable -- sum penalties given and games played
ycards <- aggregate(yellowCards ~ playerShort, sum, data = fb)   # Yellows
yrcards <- aggregate(yellowReds ~ playerShort, sum, data = fb)  # Yellow-Reds
rcards <- aggregate(redCards ~ playerShort, sum, data = fb)  # Reds
games <- aggregate(games ~ playerShort, sum, data = fb)  # Games

## Predictor variable - mean ratings to get unique value
tone1 <- aggregate(rater1 ~ playerShort, mean, data = fb)  # Rater 1
tone2 <- aggregate(rater2 ~ playerShort, mean, data = fb)  # Rater 2
```

Merging the data for analysis

Now that we've got the core indicators by player, we need to put them together in a single dataset so we can conduct our regression analysis. We could merge them one by one, but that's cumbersome:

```
## If we use 'merge(), then we have to do it one-by-one
fb2 <- merge(ycards, yrcards, by = "playerShort")
fb2 <- merge(fb2, rcards, by = "playerShort")

## and so on...
```

Instead, we'll use the `left_join()` function in the `dplyr` package (which is installed with Hadley Wickam's `tidyverse`) and `reduce()` from the `purrr` (again in `tidyverse` – isn't it such a useful package!) (with 3 r's) package to do it in one step. The `left_join()` function merges each dataframe onto whatever object comes first (i.e., read from left to right). The `reduce()` function iteratively reduces a list to a single value (e.g., try the following example in R: `1:3 %>% reduce(+)`. This "reduces" the list of numbers 1, 2, and 3 by adding them one-by-one: $1 + 2 + 3 = 6$.). Also, notice the `%>%` (pipe) operator, which is used extensively in `tidyverse` because it allows us to work through operations from left to right with each step separated by the pipe (e.g., start with some object, then do something, then something else, etc.).

Let's now merge the aggregated objects by 'playerShort', which ensures that the data is matched correctly by player name regardless of the order of each object:

```
## Merge the datasets using the 'left_join()' function from 'tidyverse'
fb.wide <- list(ycards, yrcards, rcards, games, tone1, tone2) %>%
                reduce(left_join, by = "playerShort")
glimpse(fb.wide)
```

Data analysis using linear regression (and uncertainty!)

Now we're ready to analyze the data. Our outcome will be the proportion of penalties for each player, and it might make some sense to put this on a standard metric like the *rate of penalisation* – that is, any yellow, yellow/red, or red cards per 100 games played. We also need to create an average skin tone rating between the two independent coders as our key predictor.

```
## Create tone, proportion of penalties (any cards / games played),
## and penalisation rate per 100 games(proportion * 100)
fb.wide %<>%  # Note the two-way pipe % < > % !
  mutate(
    avg.tone = (rater1 + rater2)/2 ,
```

```
    prop.cards = (yellowCards + yellowReds + redCards)/games,
    rate = prop.cards * 100
  )

## Always check your work!
fb.wide %>%
  select(avg.tone, prop.cards, rate) %>%
  summary()

## Distribution of penalisation rate
fb.wide %>%
  ggplot() +
  geom_density(aes(rate), fill = "lightblue") +
  xlab("Penalty Rate (per 100 games)") +
  ylab("Density") +
  ggtitle("Distribution of Penalty Rate")
```

Now let's regress our penalisation rate on skin tone (average rating) to test whether there appears to be any relationship between the two. The `lm()` function (for linear model) requires two arguments in the following format: `outcome ~ predictor, data = YOUR_DATA`. For example, if my data were called `df`, the outcome (the variable that I want to explain) were `dv`, and the predictor were `iv`, then you would type the following in R: `lm(dv ~ iv, data = df)`. It is generally a good idea to save your results in an object so that you can retrieve them whenever you want. Thus, your code would be something like: `reg.1 <- lm(dv ~ iv, data = df)`, and to see the results you would type `summary(reg.1)`.

What is the associated effect size of skin tone on penalties (i.e., given that skin tone ranges from 0 to 1, a 1-unit change in the predictor means moving from lightest to darkest skin tone)? Is this result statistically significant? What is the 95% confidence interval of this estimated effect (we can obtain this by using the `tab_model()` function from the `sjPlot` package)? How does incorporating uncertainty into our interpretation help you make sense of this result?

```
## Regression model - any penalty rate
model.1 <- lm(rate ~ avg.tone, data = fb.wide)
summary(model.1)

## Print results using 'sjPlot'
## They will appear as an .html output in the 'Viewer' pane
tab_model(model.1)
```

What about the effect of skin tone on receiving the most severe penalty – that is, a red card? Is there any evidence of bias among referees? Create a new outcome that is the rate of red cards per 100 games played. Check its distribution and then regress this dependent variable on skin tone.

What do you find?

```
## Rate of only red cards per 100 games
fb.wide %<>%
  mutate(
    rate2 = redCards/games * 100
  )

summary(fb.wide$rate2)

## Distribution of red card rate
fb.wide %>%
  ggplot() +
```

```
  geom_density(aes(rate2), fill = "lightblue") +
  xlab("Red Card Rate (per 100 games)") +
  ylab("Density") +
  ggtitle("Distribution of Red Card Rate")

## Regression model 2 - red card rate
model.2 <- lm(rate2 ~ avg.tone, data = fb.wide)
summary(model.2)

tab_model(model.2)
```

Additional analyses

Our simple model may not be a good approximation of reality because it probably misses many important predictors (and likely has issues because of the skewed nature of the dependent variable, but that's for us to deal with on a different day). What other variables would you include in a more thorough examination? What do those updated results tell you?

To do additional analyses, we'll need to merge other predictors into our `fb.wide2` aggregated data. We can do that the same way we did above, or we can use `dplyr` from `tidyverse` to do everything in one efficient chunk of code.

```
fb.wide2 <- fb %>%
    group_by(playerShort) %>%  # Group by each player
    summarise(  # summarise will create new object
      country = unique(leagueCountry),
      club = unique(club),
      bday = unique(dmy(birthday)), # Use lubridate to date format
      age = 2013 - year(bday),  # Subtract from 2013 to find age
      position = unique(position),
      games = sum(games),
      victories = sum(victories),
      ties = sum(ties),
      defeats = sum(defeats),
      goals = sum(goals),
      yellowCards = sum(yellowCards),
      yellowReds = sum(yellowReds),
      redCards = sum(redCards),
      rater1 = unique(rater1),
      rater2 = unique(rater2),
      ## Add in calculated variables from above
      avg.tone = (rater1 + rater2)/2 ,
      prop.cards = (yellowCards + yellowReds + redCards)/games,
      rate = prop.cards * 100,
      rate2 = redCards/games * 100
              )
```

What else might we include in a model as additional predictors? How would we include nominal (i.e., categorical, non-continuous) predictors (e.g., `leagueCountry`, `position`, etc.)? Make sure you interpret your results fully (e.g., each coefficient, statistical significance, confidence intervals, model R-squared, etc.). Enjoy!